TECNOLÓGICO DE ESTUDIOS SUPERIORES DE ECATEPEC

DIVISIÓN DE INGENIERÍA EN SISTEMAS COMPUTACIONALES

PROGRAMACIÓN DE BASE DE DATOS

TAREA 5: EL MODELO CLIENTE – SERVIDOR

PROFESORA:

ALEJANDRA GONZALEZ SÁNCHEZ

NOMBRE:

MONTERRUBIO LÓPEZ LUIS GAEL

GRUPO: 5701

FECHA: 30-10-26

1. Introducción y objetivo

Este documento explica el modelo cliente-servidor, sus componentes, su funcionamiento y su aplicación en sistemas reales. Después describe el diseño y la implementación de un sistema de base de datos con esta arquitectura para el proyecto IoT Analytics Dashboard de NexoDev Solutions.

El sistema recopila lecturas de sensores IoT simulados. Las lecturas viajan por RabbitMQ, se guardan en un servidor PostgreSQL y se consultan desde un cliente de línea de comandos y desde un tablero de Grafana. Todo se ejecuta con Docker.

Repositorio del proyecto: https://github.com/luisgael-ml/iot-cliente-servidor

Competencia específica: elaborar un sistema de base de datos con arquitectura cliente-servidor (clave de actividad BDM-1301T02CE01A01).

Objetivos específicos:

1. Investigar el modelo cliente-servidor y documentar sus componentes y funcionamiento con diagramas de arquitectura.
2. Diseñar la estructura de la base de datos en PostgreSQL para un sistema IoT.
3. Configurar un servidor de base de datos con Docker y un cliente que realice consultas y manipulaciones de datos.
4. Simular la recopilación de datos IoT y demostrar la interacción cliente-servidor con RabbitMQ y Grafana.

2. Marco teórico: el modelo cliente-servidor

2.1 ¿Qué es el modelo cliente-servidor?

Es una arquitectura de software que reparte el trabajo entre dos tipos de programas: los **clientes**, que solicitan un servicio, y los **servidores**, que lo proporcionan. El cliente envía una petición por la red, el servidor la procesa y devuelve una respuesta.

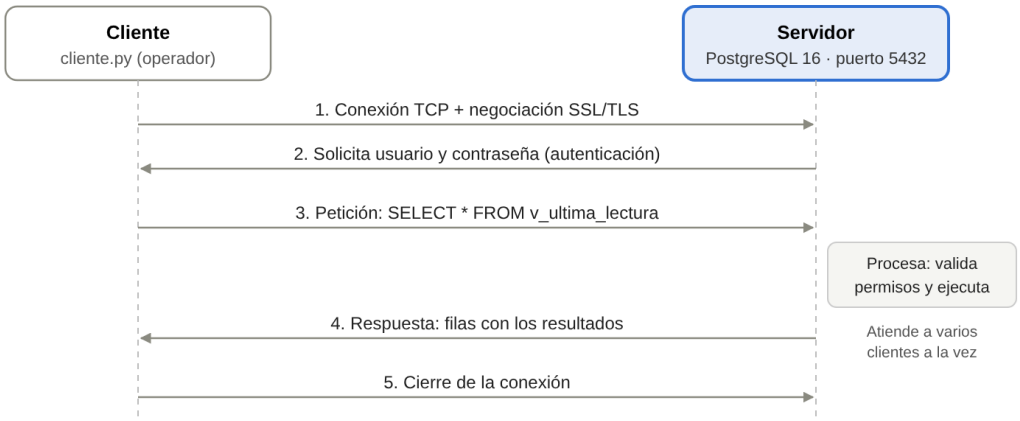
Un mismo servidor atiende a muchos clientes a la vez, y los clientes no se comunican entre sí: todo pasa por el servidor. Esto centraliza los datos, la seguridad y las reglas del negocio en un solo lugar.

2.2 Componentes y su función

COMPONENTE	FUNCIÓN	EN ESTE PROYECTO
Cliente	Inicia la comunicación: envía peticiones y presenta los resultados al usuario	cliente.py, simulador, consumidor y Grafana
Servidor	Espera peticiones, las procesa y responde; administra los recursos compartidos	PostgreSQL 16 (datos) y RabbitMQ (mensajes)
Red	Medio por el que viajan peticiones y respuestas	Red interna de Docker Compose
Protocolo	Reglas que definen el formato de los mensajes	Protocolo frontend/backend de PostgreSQL y AMQP
Middleware	Software intermedio que conecta clientes y servidores	Drivers psycopg y pika, y el intermediario RabbitMQ
Puerto	Número donde el servidor escucha las conexiones	5432 (PostgreSQL), 5672 (AMQP), 3000 (Grafana)

2.3 Funcionamiento: el ciclo petición-respuesta

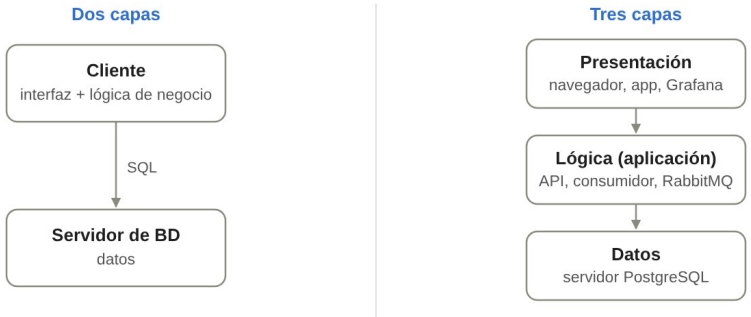
Toda interacción sigue el mismo ciclo. El diagrama muestra una consulta real del cliente de este proyecto al servidor PostgreSQL:



Ciclo petición-respuesta entre el cliente de consultas y el servidor PostgreSQL.

1. **Conexión:** el cliente abre una conexión TCP al puerto del servidor y negocian el cifrado SSL/TLS.
2. **Autenticación:** el servidor verifica el usuario y la contraseña, y le asigna sus permisos.
3. **Petición:** el cliente envía una sentencia SQL.
4. **Procesamiento:** el servidor valida los permisos, ejecuta la consulta y mientras tanto atiende a otros clientes. PostgreSQL crea un proceso por cada conexión.
5. **Respuesta:** el servidor devuelve las filas o el número de filas afectadas.
6. **Cierre:** el cliente termina la sesión y el servidor libera los recursos.

2.4 Arquitecturas de dos, tres y n capas



Comparación entre las arquitecturas de dos y tres capas.

ARQUITECTURA	CÓMO SE ORGANIZA	VENTAJA	LIMITACIÓN
Dos capas	El cliente contiene la interfaz y la lógica; se conecta directo al servidor de BD	Sencilla y rápida de construir	Cada cliente necesita credenciales de la BD; difícil de escalar
Tres capas	Presentación, lógica de negocio y datos separadas	La lógica se cambia sin tocar a los clientes; más segura	Más componentes que mantener

ARQUITECTURA	CÓMO SE ORGANIZA	VENTAJA	LIMITACIÓN
N capas	Se agregan capas especializadas: colas de mensajes, cachés, microservicios	Escala por partes y tolera fallas	Mayor complejidad de despliegue

El sistema de este proyecto es de **n capas**: los sensores no escriben directo en la base, sino que pasan por RabbitMQ y un consumidor.

2.5 Tipos de servidores

TIPO	QUÉ OFRECE	EJEMPLOS
Base de datos	Almacenamiento y consulta de datos con SQL	PostgreSQL, MySQL, SQL Server, Oracle
Web	Páginas y APIs por HTTP	Nginx, Apache, IIS
Aplicaciones	Ejecuta la lógica de negocio	Tomcat, Kestrel (.NET), Node.js
Mensajería	Recibe, guarda y entrega mensajes entre programas	RabbitMQ, Kafka
Archivos	Almacena y comparte archivos	Servidores SMB, FTP, almacenamiento en la nube
Correo	Envía y recibe correo electrónico	Servidores SMTP e IMAP

2.6 Características

- **Centralización:** los datos y las reglas viven en el servidor; todos los clientes ven la misma información.
- **Concurrencia:** el servidor atiende muchas peticiones simultáneas y controla que no se interfieran (transacciones y bloqueos).
- **Escalabilidad:** se pueden agregar clientes sin cambiar el servidor, o mejorar el servidor sin cambiar a los clientes.
- **Seguridad centralizada:** autenticación, permisos y cifrado se configuran en un solo lugar.
- **Transparencia de ubicación:** el cliente solo conoce la dirección y el puerto del servidor, no cómo está construido.
- **Heterogeneidad:** clientes en distintos lenguajes y sistemas operativos usan el mismo servidor. En este proyecto, Python y Grafana; en las tareas anteriores, Java y C#.

2.7 Ventajas y desventajas

VENTAJAS	DESVENTAJAS
Datos consistentes en un solo lugar	El servidor es un punto único de falla si no hay réplicas
Control de acceso centralizado	Muchas peticiones simultáneas pueden saturarlo
Mantenimiento más sencillo: se actualiza el servidor y todos los clientes se benefician	Depende de la red: sin conexión, el cliente no funciona
Clientes ligeros: el procesamiento pesado ocurre en el servidor	Requiere hardware y administración del servidor
Respaldos y recuperación centralizados	Escalar el servidor verticalmente tiene un límite de costo

2.8 Cliente-servidor frente a peer-to-peer

ASPECTO	CLIENTE-SERVIDOR	PEER-TO-PEER (P2P)
Roles	Fijos: unos piden, otros sirven	Cada nodo es cliente y servidor a la vez
Datos	Centralizados en el servidor	Distribuidos entre los nodos
Control y seguridad	Centralizados, fáciles de administrar	Descentralizados, más difíciles de controlar
Falla de un nodo	Si cae el servidor, el servicio se detiene	El resto de los nodos sigue funcionando
Ejemplos	Bases de datos, banca, correo, web	BitTorrent, redes blockchain

2.9 Aplicación en sistemas reales

SISTEMA	CLIENTE	SERVIDOR	PROTOCOLO
Navegación web	Navegador (Chrome, Edge)	Servidor web (Nginx, Apache)	HTTP/HTTPS
Banca en línea y cajeros	App móvil o cajero automático	Servidores del banco con su base de datos	HTTPS y redes bancarias privadas
Correo electrónico	Outlook, Gmail	Servidores de correo	SMTP, IMAP
ERP empresarial	SAP GUI o navegador	Servidor de aplicaciones SAP y su base de datos	Protocolos propios de SAP y HTTPS
Resolución de nombres	El sistema operativo	Servidores DNS	DNS
Videojuegos en línea	El juego en cada consola o PC	Servidores de partidas	TCP/UDP propios
IoT Analytics Dashboard (este proyecto)	Simulador, consumidor, cliente y Grafana	PostgreSQL y RabbitMQ	Protocolo de PostgreSQL y AMQP

2.10 PostgreSQL como servidor de base de datos

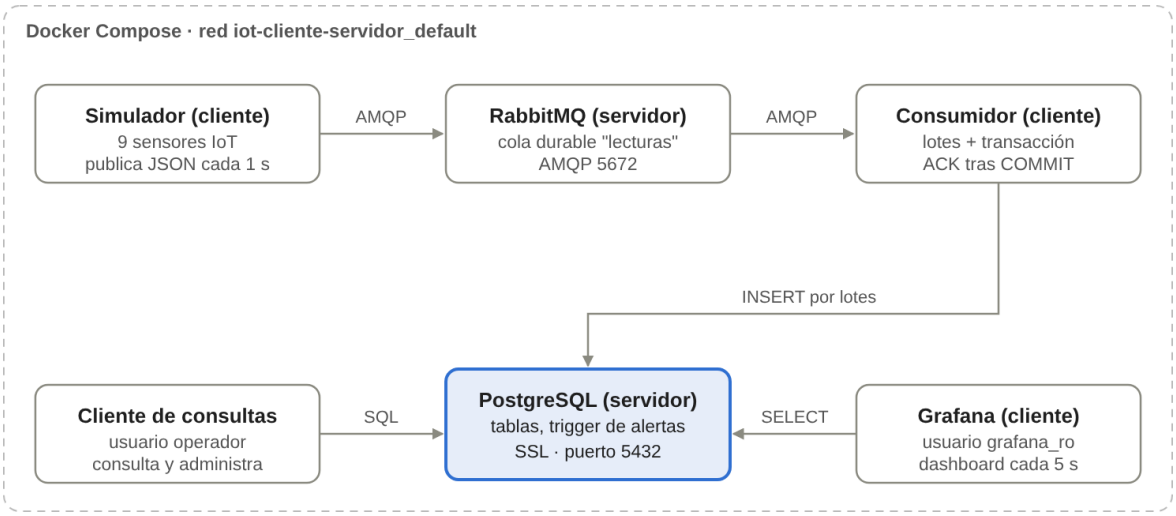
PostgreSQL sigue el modelo cliente-servidor. Un proceso principal escucha en el puerto 5432; cuando llega una conexión, crea un proceso dedicado para ese cliente, que vive mientras dure la sesión. Así varios clientes trabajan al mismo tiempo sin bloquearse entre sí.

Los clientes pueden ser la terminal `psql`, una herramienta gráfica o cualquier programa que use un driver: JDBC en Java (Tarea 3), Npgsql en C# (Tarea 4) y psycopg en Python (esta tarea). Todos hablan el mismo protocolo frontend/backend.

3. Diseño del sistema IoT Analytics Dashboard

3.1 Arquitectura

El sistema tiene dos servidores y cuatro clientes. Cada componente corre en su propio contenedor y se comunica por la red interna de Docker usando el nombre del servicio.



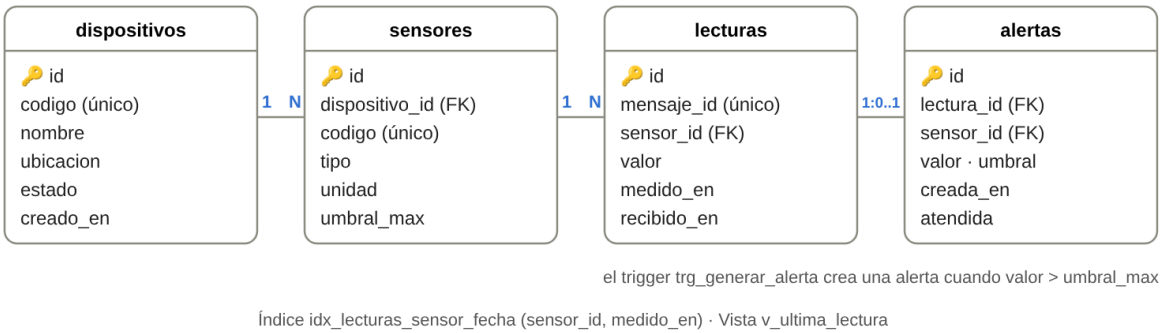
Arquitectura cliente-servidor del IoT Analytics Dashboard.

COMPONENTE	ROL	SE CONECTA A	USUARIO DE BD	PUERTO EN LA PC
PostgreSQL 16	Servidor de datos	—	postgres (administrador)	5434
RabbitMQ 3.13	Servidor de mensajes	—	—	5673 y 15673 (panel)
Simulador	Cliente productor	RabbitMQ	—	—
Consumidor	Cliente de ingesta	RabbitMQ y PostgreSQL	ingesta	—
Cliente de consultas	Cliente del operador	PostgreSQL	operador	—
Grafana 11.2	Cliente de visualización	PostgreSQL	grafana_ro	3001

El flujo de una lectura es: el simulador la publica en la cola; RabbitMQ la guarda hasta que el consumidor la toma; el consumidor la inserta en PostgreSQL dentro de un lote; el trigger del servidor genera una alerta si supera el umbral; y Grafana y el operador la consultan.

3.2 Modelo de datos

La base de datos separa los dispositivos, sus sensores, el historial de lecturas y las alertas. Las lecturas solo reciben inserciones, lo que mantiene el historial completo.



Modelo entidad-relación de la base de datos iotanalytics.

TABLA	CONTENIDO	DECISIONES DE DISEÑO
dispositivos	Máquinas de la planta	Código único y estado restringido con CHECK
sensores	Sensores de cada dispositivo	Tipo, unidad y umbral de alerta por sensor
lecturas	Historial de mediciones	mensaje_id único contra duplicados; índice por sensor y fecha
alertas	Lecturas que superaron el umbral	Las genera el servidor con un trigger, no el cliente

3.3 Lógica del lado del servidor

Parte de las reglas vive en el servidor para que se cumplan sin importar qué cliente inserte los datos:

- **Trigger `trg_generar_alerta`:** después de cada inserción en `lecturas`, compara el valor con el umbral del sensor y crea la alerta.
- **Vista `v_ultima_lectura`:** entrega la última lectura de cada sensor; el cliente y Grafana la consultan sin repetir la lógica.
- **Restricciones:** llaves foráneas, valores únicos y CHECK garantizan datos válidos.

3.4 Seguridad

USUARIO	LO USA	PERMISOS
ingesta	Consumidor	Solo insertar lecturas y alertas, y leer sensores
operador	Cliente de consultas	Consultar todo, registrar dispositivos, cambiar umbrales, atender alertas y borrar lecturas antiguas
grafana_ro	Grafana	Solo lectura

Además, todas las conexiones a PostgreSQL van cifradas con SSL (`sslmode=require`), y las contraseñas se pasan como variables de entorno, no en el código.

4. Implementación

4.1 Estructura del proyecto

```
iot-cliente-servidor/
├── docker-compose.yml          → servidores y clientes
├── servidor/init.sql           → esquema, trigger, vista, usuarios y datos
├── clientes/
│   ├── Dockerfile              → imagen común de los clientes (Python 3.12)
│   ├── requirements.txt        → psycopg y pika
│   ├── conexiones.py           → conexión a PostgreSQL y RabbitMQ
│   ├── simulador.py            → cliente 1: sensores IoT
│   ├── consumidor.py           → cliente 2: ingesta por lotes
│   └── cliente.py              → cliente 3: consultas del operador
└── grafana/                    → fuente de datos y dashboard
```

4.2 Orquestación con Docker Compose

```
# Sistema cliente-servidor del IoT Analytics Dashboard (NexoDev Solutions)
name: iot-cliente-servidor

services:
  # ---------------- SERVIDOR DE BASE DE DATOS ----------------
  postgres:
    image: postgres:16
```

```
  container_name: iot-servidor-bd
  environment:
    POSTGRES_PASSWORD: ${DB_PASSWORD:-admin123}
    POSTGRES_DB: iotanalytics
  command: >
    -c ssl=on
    -c ssl_cert_file=/etc/ssl/certs/ssl-cert-snakeoil.pem
    -c ssl_key_file=/etc/ssl/private/ssl-cert-snakeoil.key
  ports:
    - "5434:5432"          # 5434 en la PC para no chocar con otros PostgreSQL
  volumes:
    - ./servidor/init.sql:/docker-entrypoint-initdb.d/init.sql
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres -d iotanalytics"]
    interval: 3s
    retries: 10

# ---------------- INTERMEDIARIO DE MENSAJES ----------------
rabbitmq:
  image: rabbitmq:3.13-management
  container_name: iot-rabbitmq
  hostname: iot-rabbitmq   # nombre fijo del nodo de RabbitMQ
  environment:
    # La cookie de Erlang se pasa como argumento: así RabbitMQ no necesita leer
    # /var/lib/rabbitmq/.erlang.cookie, que fallaba con "eacces" en Docker Desktop
    RABBITMQ_SERVER_ADDITIONAL_ERL_ARGS: "-setcookie iotanalyticscookie"
  tmpfs:
    - /var/lib/rabbitmq:mode=1777
  ports:
    - "5673:5672"          # AMQP
    - "15673:15672"        # panel web: http://localhost:15673 (guest / guest)
  healthcheck:
    # Comprueba que el puerto AMQP acepte conexiones (no usa la cookie)
    test: ["CMD-SHELL", "bash -c '</dev/tcp/localhost/5672'"]
    interval: 5s
    retries: 30

# ---------------- CLIENTES ----------------
# Cliente 1: simula los sensores y publica lecturas en RabbitMQ
simulador:
  build: ./clientes
  image: iot-clientes:1.0
  command: python simulador.py
  environment:
    MQ_HOST: rabbitmq
    INTERVALO: "1"
  depends_on:
    rabbitmq:
      condition: service_healthy

# Cliente 2: consume la cola y guarda las lecturas en el servidor por lotes
consumidor:
  build: ./clientes
  image: iot-clientes:1.0
  command: python consumidor.py
  environment:
    MQ_HOST: rabbitmq
    DB_HOST: postgres
    DB_USER: ingesta
    DB_PASSWORD: ingesta123
```

```
    depends_on:
      postgres:
        condition: service_healthy
      rabbitmq:
        condition: service_healthy
      simulador:
        condition: service_started

  # Cliente 3: consultas del operador (se ejecuta a demanda con docker compose run)
  cliente:
    build: ./clientes
    image: iot-clientes:1.0
    command: python cliente.py
    environment:
      DB_HOST: postgres
      DB_USER: operador
      DB_PASSWORD: operador123
    stdin_open: true
    tty: true
    profiles: ["cliente"]
    depends_on:
      postgres:
        condition: service_healthy

  # Cliente 4: Grafana, visualización (solo lectura)
  grafana:
    image: grafana/grafana:11.2.0
    container_name: iot-grafana
    ports:
      - "3001:3000"          # http://localhost:3001 (admin / admin)
    environment:
      GF_SECURITY_ADMIN_PASSWORD: admin
      GF_AUTH_ANONYMOUS_ENABLED: "true"
      GF_AUTH_ANONYMOUS_ORG_ROLE: Viewer
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning
      - ./grafana/dashboards:/var/lib/grafana/dashboards
    depends_on:
      postgres:
        condition: service_healthy
```

4.3 Servidor de base de datos

```
-- ========================================================================
-- Servidor de base de datos: IoT Analytics Dashboard (NexoDev Solutions)
-- Esquema relacional para dispositivos, sensores, lecturas y alertas
-- ========================================================================

-- Dispositivos físicos instalados en la planta
CREATE TABLE dispositivos (
    id         SERIAL PRIMARY KEY,
    codigo     VARCHAR(20)  NOT NULL UNIQUE,
    nombre     VARCHAR(60)  NOT NULL,
    ubicacion  VARCHAR(60)  NOT NULL,
    estado     VARCHAR(15)  NOT NULL DEFAULT 'activo'
               CHECK (estado IN ('activo', 'mantenimiento', 'inactivo')),
    creado_en  TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

-- Cada dispositivo tiene uno o varios sensores
```

```
CREATE TABLE sensores (
    id              SERIAL PRIMARY KEY,
    dispositivo_id  INT          NOT NULL REFERENCES dispositivos(id) ON DELETE CASCADE,
    codigo          VARCHAR(20)  NOT NULL UNIQUE,
    tipo            VARCHAR(20)  NOT NULL,   -- temperatura, vibracion, energia
    unidad          VARCHAR(10)  NOT NULL,   -- °C, mm/s, kWh
    umbral_max      NUMERIC(10,2)            -- valor a partir del cual se genera una alerta
);

-- Historial de lecturas (solo inserciones)
CREATE TABLE lecturas (
    id          BIGSERIAL PRIMARY KEY,
    mensaje_id  UUID          NOT NULL UNIQUE,    -- evita duplicados si RabbitMQ reenvía un mensaje
    sensor_id   INT           NOT NULL REFERENCES sensores(id) ON DELETE CASCADE,
    valor       NUMERIC(10,2) NOT NULL,
    medido_en   TIMESTAMPTZ   NOT NULL,           -- hora en que el sensor tomó la lectura
    recibido_en TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);
-- Índice para consultar rápido las lecturas de un sensor en un rango de tiempo
CREATE INDEX idx_lecturas_sensor_fecha ON lecturas (sensor_id, medido_en DESC);

-- Alertas generadas por el propio servidor cuando una lectura supera el umbral
CREATE TABLE alertas (
    id          SERIAL PRIMARY KEY,
    lectura_id  BIGINT        NOT NULL REFERENCES lecturas(id) ON DELETE CASCADE,
    sensor_id   INT           NOT NULL REFERENCES sensores(id) ON DELETE CASCADE,
    valor       NUMERIC(10,2) NOT NULL,
    umbral      NUMERIC(10,2) NOT NULL,
    creada_en   TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
    atendida    BOOLEAN       NOT NULL DEFAULT FALSE
);

-- ---------------------------------------------------------------------
-- Lógica del lado del servidor: trigger que crea alertas automáticamente
-- ---------------------------------------------------------------------
CREATE FUNCTION generar_alerta() RETURNS TRIGGER AS $$
DECLARE
    v_umbral NUMERIC(10,2);
BEGIN
    SELECT umbral_max INTO v_umbral FROM sensores WHERE id = NEW.sensor_id;
    IF v_umbral IS NOT NULL AND NEW.valor > v_umbral THEN
        INSERT INTO alertas (lectura_id, sensor_id, valor, umbral)
        VALUES (NEW.id, NEW.sensor_id, NEW.valor, v_umbral);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_generar_alerta
AFTER INSERT ON lecturas
FOR EACH ROW EXECUTE FUNCTION generar_alerta();

-- Vista con la última lectura de cada sensor (usada por el cliente y Grafana)
CREATE VIEW v_ultima_lectura AS
SELECT DISTINCT ON (s.id)
       d.codigo AS dispositivo, d.ubicacion, s.codigo AS sensor, s.tipo, s.unidad,
       l.valor, s.umbral_max, l.medido_en
FROM sensores s
JOIN dispositivos d ON d.id = s.dispositivo_id
LEFT JOIN lecturas l ON l.sensor_id = s.id
```

```
ORDER BY s.id, l.medido_en DESC NULLS LAST;

-- ---------------------------------------------------------------------
-- Usuarios con permisos mínimos (cada cliente solo puede lo que necesita)
-- ---------------------------------------------------------------------
-- Consumidor de RabbitMQ: solo inserta lecturas
CREATE ROLE ingesta LOGIN PASSWORD 'ingesta123';
GRANT SELECT ON sensores TO ingesta;
GRANT INSERT ON lecturas, alertas TO ingesta;
GRANT USAGE ON SEQUENCE lecturas_id_seq, alertas_id_seq TO ingesta;
GRANT SELECT ON lecturas TO ingesta;   -- necesario para ON CONFLICT

-- Cliente de consultas (operador de planta): consulta y administra, pero no borra dispositivos
CREATE ROLE operador LOGIN PASSWORD 'operador123';
GRANT SELECT ON ALL TABLES IN SCHEMA public TO operador;
GRANT INSERT, UPDATE ON dispositivos, sensores TO operador;
GRANT UPDATE (atendida) ON alertas TO operador;
GRANT DELETE ON lecturas, alertas TO operador;
GRANT USAGE ON SEQUENCE dispositivos_id_seq, sensores_id_seq TO operador;

-- Grafana: solo lectura
CREATE ROLE grafana_ro LOGIN PASSWORD 'grafana123';
GRANT SELECT ON ALL TABLES IN SCHEMA public TO grafana_ro;

-- ---------------------------------------------------------------------
-- Datos iniciales: 3 dispositivos con 3 sensores cada uno
-- ---------------------------------------------------------------------
INSERT INTO dispositivos (codigo, nombre, ubicacion) VALUES
('PRE-03', 'Prensa hidráulica 3', 'Línea 1'),
('TOR-01', 'Torno CNC 1',         'Línea 2'),
('EMB-02', 'Embotelladora 2',     'Línea 3');

INSERT INTO sensores (dispositivo_id, codigo, tipo, unidad, umbral_max) VALUES
(1, 'PRE03-TEMP', 'temperatura', '°C',   85.00),
(1, 'PRE03-VIB',  'vibracion',   'mm/s',  7.00),
(1, 'PRE03-ENE',  'energia',     'kWh',  30.00),
(2, 'TOR01-TEMP', 'temperatura', '°C',   80.00),
(2, 'TOR01-VIB',  'vibracion',   'mm/s',  6.00),
(2, 'TOR01-ENE',  'energia',     'kWh',  25.00),
(3, 'EMB02-TEMP', 'temperatura', '°C',   75.00),
(3, 'EMB02-VIB',  'vibracion',   'mm/s',  5.00),
(3, 'EMB02-ENE',  'energia',     'kWh',  20.00);
```

4.4 Conexiones de los clientes

```
"""Conexiones compartidas por los clientes: PostgreSQL (servidor de BD) y RabbitMQ."""
import os
import time

import pika
import psycopg

COLA = "lecturas"   # cola durable donde los sensores publican sus lecturas


def conectar_bd(usuario_env="DB_USER", password_env="DB_PASSWORD", intentos=30):
    """Abre una conexión cifrada (SSL) con el servidor PostgreSQL, reintentando mientras arranca."""
    params = dict(
        host=os.getenv("DB_HOST", "localhost"),
        port=int(os.getenv("DB_PORT", "5432")),
```

```
        dbname=os.getenv("DB_NAME", "iotanalytics"),
        user=os.getenv(usuario_env, "postgres"),
        password=os.environ[password_env],          # la contraseña nunca está en el código
        sslmode=os.getenv("DB_SSLMODE", "require"),
    )
    for i in range(1, intentos + 1):
        try:
            return psycopg.connect(**params)
        except psycopg.OperationalError as e:
            if i == intentos:
                raise
            print(f"Esperando a PostgreSQL ({i}/{intentos}): {str(e).strip()}")
            time.sleep(2)


def conectar_rabbitmq(intentos=30):
    """Abre una conexión con RabbitMQ y declara la cola durable de lecturas."""
    credenciales = pika.PlainCredentials(os.getenv("MQ_USER", "guest"), os.getenv("MQ_PASSWORD",
"guest"))
    params = pika.ConnectionParameters(host=os.getenv("MQ_HOST", "localhost"), credentials=credenciales,
                                       heartbeat=60)
    for i in range(1, intentos + 1):
        try:
            conexion = pika.BlockingConnection(params)
            canal = conexion.channel()
            canal.queue_declare(queue=COLA, durable=True)   # la cola sobrevive a un reinicio
            return conexion, canal
        except pika.exceptions.AMQPConnectionError:
            if i == intentos:
                raise
            print(f"Esperando a RabbitMQ ({i}/{intentos})")
            time.sleep(2)
```

4.5 Cliente 1: simulador de sensores

Genera cada segundo una lectura por sensor, con picos ocasionales para probar las alertas, y la publica en la cola durable `lecturas`.

```
"""
CLIENTE 1 · Simulador de sensores IoT (productor).
Cada segundo genera una lectura por sensor y la publica en RabbitMQ como JSON.
No conoce la base de datos: solo envía mensajes a la cola.
"""
import json
import os
import random
import time
import uuid
from datetime import datetime, timezone

import pika

from conexiones import COLA, conectar_rabbitmq

# sensor -> (valor base, variación, probabilidad de pico que supera el umbral)
SENSORES = {
    "PRE03-TEMP": (70, 3, 0.04), "PRE03-VIB": (4.5, 0.6, 0.04), "PRE03-ENE": (22, 2, 0.03),
    "TOR01-TEMP": (65, 3, 0.04), "TOR01-VIB": (3.8, 0.5, 0.04), "TOR01-ENE": (18, 2, 0.03),
    "EMB02-TEMP": (60, 3, 0.04), "EMB02-VIB": (3.0, 0.4, 0.04), "EMB02-ENE": (14, 2, 0.03),
}
```

```
def generar(codigo):
    base, variacion, prob_pico = SENSORES[codigo]
    valor = random.gauss(base, variacion)
    if random.random() < prob_pico:
        valor = base * 1.35                      # pico anómalo para probar las alertas
    return {
        "mensaje_id": str(uuid.uuid4()),
        "sensor": codigo,
        "valor": round(valor, 2),
        "medido_en": datetime.now(timezone.utc).isoformat(),
    }


def main():
    intervalo = float(os.getenv("INTERVALO", "1"))
    duracion = int(os.getenv("DURACION", "0"))   # 0 = sin límite
    conexion, canal = conectar_rabbitmq()
    print(f"Simulador conectado a RabbitMQ. Publicando en la cola '{COLA}' cada {intervalo}s...")
    inicio, enviados = time.time(), 0
    try:
        while duracion == 0 or time.time() - inicio < duracion:
            for codigo in SENSORES:
                lectura = generar(codigo)
                canal.basic_publish(
                    exchange="", routing_key=COLA, body=json.dumps(lectura),
                    properties=pika.BasicProperties(delivery_mode=2, content_type="application/json"),
                )
                enviados += 1
            print(f"-> {enviados} lecturas publicadas (última: {lectura['sensor']} =
{lectura['valor']})")
            time.sleep(intervalo)
    finally:
        conexion.close()
        print(f"Simulador detenido. Total publicado: {enviados}")


if __name__ == "__main__":
    main()
```

4.6 Cliente 2: consumidor por lotes

Agrupa los mensajes en lotes de hasta 100 o de 1 segundo, los guarda en una transacción y confirma los mensajes con un ACK múltiple solo después del COMMIT. Si falla antes, los mensajes regresan a la cola, y `ON CONFLICT DO NOTHING` evita duplicados al reprocesarlos.

```
"""
CLIENTE 2 · Consumidor de RabbitMQ (servicio de ingesta).
Lee las lecturas de la cola, las agrupa en lotes y las guarda en PostgreSQL
dentro de una transacción. Solo confirma (ACK) los mensajes después del COMMIT.
"""
import json
import os
import time

from conexiones import COLA, conectar_bd, conectar_rabbitmq

TAM_LOTE = int(os.getenv("TAM_LOTE", "100"))   # máximo de mensajes por lote
ESPERA = float(os.getenv("ESPERA_LOTE", "1"))  # segundos máximos antes de guardar un lote incompleto
```

```
SQL_INSERT = """
    INSERT INTO lecturas (mensaje_id, sensor_id, valor, medido_en)
    SELECT %(mensaje_id)s, s.id, %(valor)s, %(medido_en)s
    FROM sensores s WHERE s.codigo = %(sensor)s
    ON CONFLICT (mensaje_id) DO NOTHING      -- si el mensaje ya se guardó, se ignora
"""


def guardar_lote(bd, lote):
    """Guarda el lote completo en una sola transacción. Devuelve las filas insertadas."""
    with bd.transaction():                   # COMMIT al salir; ROLLBACK si hay error
        with bd.cursor() as cur:
            cur.executemany(SQL_INSERT, lote)
            return cur.rowcount


def main():
    bd = conectar_bd(usuario_env="DB_USER", password_env="DB_PASSWORD")
    conexion, canal = conectar_rabbitmq()
    canal.basic_qos(prefetch_count=TAM_LOTE * 2)    # cuántos mensajes recibe sin confirmar
    print(f"Consumidor listo. Lotes de hasta {TAM_LOTE} mensajes o {ESPERA}s.")

    lote, ultima_etiqueta, total, inicio_lote = [], None, 0, 0.0
    for metodo, _props, cuerpo in canal.consume(COLA, inactivity_timeout=ESPERA):
        if metodo is not None:
            if not lote:
                inicio_lote = time.time()
            lote.append(json.loads(cuerpo))
            ultima_etiqueta = metodo.delivery_tag
        # Guarda cuando el lote se llena o cuando ya pasó el tiempo máximo de espera
        if lote and (len(lote) >= TAM_LOTE or time.time() - inicio_lote >= ESPERA):
            insertadas = guardar_lote(bd, lote)
            canal.basic_ack(delivery_tag=ultima_etiqueta, multiple=True)   # ACK múltiple tras el COMMIT
            total += insertadas
            print(f"Lote guardado: {len(lote)} mensajes, {insertadas} insertados (total: {total})")
            lote = []


if __name__ == "__main__":
    main()
```

4.7 Cliente 3: consultas del operador

```
"""
CLIENTE 3 · Cliente de consultas (operador de planta).
Envía peticiones SQL al servidor PostgreSQL y muestra las respuestas:
consultas, altas, actualizaciones y bajas.
  python cliente.py          -> menú interactivo
  python cliente.py --demo   -> ejecuta las consultas principales sin interacción
"""
import sys

from conexiones import conectar_bd


def tabla(cur, sql, params=None):
    """Envía una consulta al servidor e imprime el resultado como tabla."""
    cur.execute(sql, params)
    columnas = [c.name for c in cur.description]
```

```
    filas = cur.fetchall()
    anchos = [max(len(str(c)), *(len(str(f[i])) for f in filas)) if filas else len(c)
              for i, c in enumerate(columnas)]
    print("  ".join(str(c).upper().ljust(a) for c, a in zip(columnas, anchos)))
    for f in filas:
        print("  ".join(str(v).ljust(a) for v, a in zip(f, anchos)))
    print(f"({len(filas)} filas)")


CONSULTAS = {
    "dispositivos": """
        SELECT d.codigo, d.nombre, d.ubicacion, d.estado, COUNT(s.id) AS sensores
        FROM dispositivos d LEFT JOIN sensores s ON s.dispositivo_id = d.id
        GROUP BY d.id ORDER BY d.codigo""",
    "ultimas": """
        SELECT dispositivo, sensor, tipo, valor, unidad, umbral_max,
               to_char(medido_en AT TIME ZONE 'America/Mexico_City', 'HH24:MI:SS') AS hora
        FROM v_ultima_lectura ORDER BY dispositivo, sensor""",
    "resumen": """
        SELECT s.codigo AS sensor, COUNT(*) AS lecturas,
               ROUND(AVG(l.valor), 2) AS promedio, MIN(l.valor) AS minimo, MAX(l.valor) AS maximo
        FROM lecturas l JOIN sensores s ON s.id = l.sensor_id
        WHERE l.medido_en > NOW() - INTERVAL '1 hour'
        GROUP BY s.codigo ORDER BY s.codigo""",
    "alertas": """
        SELECT a.id, s.codigo AS sensor, a.valor, a.umbral,
               to_char(a.creada_en AT TIME ZONE 'America/Mexico_City', 'HH24:MI:SS') AS hora
        FROM alertas a JOIN sensores s ON s.id = a.sensor_id
        WHERE NOT a.atendida ORDER BY a.creada_en DESC LIMIT 10""",
}


def demo(bd):
    with bd.cursor() as cur:
        for titulo, clave in [("Dispositivos registrados", "dispositivos"),
                              ("Última lectura de cada sensor", "ultimas"),
                              ("Resumen de la última hora", "resumen"),
                              ("Alertas pendientes (generadas por el trigger del servidor)", "alertas")]:
            print(f"\n--- {titulo} ---")
            tabla(cur, CONSULTAS[clave])


def menu(bd):
    opciones = """
1. Ver dispositivos
2. Última lectura de cada sensor
3. Resumen de la última hora (promedio, mínimo, máximo)
4. Alertas pendientes
5. Marcar alerta como atendida
6. Registrar dispositivo nuevo
7. Cambiar umbral de un sensor
8. Eliminar lecturas con más de N minutos
0. Salir"""
    while True:
        print(opciones)
        op = input("Opción: ").strip()
        try:
            with bd.cursor() as cur:
                if op in ("1", "2", "3", "4"):
                    tabla(cur, CONSULTAS[["dispositivos", "ultimas", "resumen", "alertas"][int(op) - 1]])
```

```
                elif op == "5":
                    cur.execute("UPDATE alertas SET atendida = TRUE WHERE id = %s", (int(input("ID de la
alerta: ")),))
                    print("Alerta atendida." if cur.rowcount else "ID no encontrado.")
                elif op == "6":
                    cur.execute("INSERT INTO dispositivos (codigo, nombre, ubicacion) VALUES (%s, %s, %s)
RETURNING id",
                                (input("Código: "), input("Nombre: "), input("Ubicación: ")))
                    print(f"Dispositivo registrado con id {cur.fetchone()[0]}.")
                elif op == "7":
                    cur.execute("UPDATE sensores SET umbral_max = %s WHERE codigo = %s",
                                (float(input("Nuevo umbral: ")), input("Código del sensor: ")))
                    print("Umbral actualizado." if cur.rowcount else "Sensor no encontrado.")
                elif op == "8":
                    cur.execute("DELETE FROM lecturas WHERE medido_en < NOW() - make_interval(mins =>
%s)",
                                (int(input("Minutos: ")),))
                    print(f"{cur.rowcount} lecturas eliminadas.")
                elif op == "0":
                    print("Hasta luego.")
                    return
                else:
                    print("Opción inválida.")
            bd.commit()
        except Exception as e:          # errores de SQL o de conexión no cierran el cliente
            bd.rollback()
            print(f"Error: {e}")


def main():
    bd = conectar_bd(usuario_env="DB_USER", password_env="DB_PASSWORD")
    with bd.cursor() as cur:
        cur.execute("SELECT current_user, ssl, version FROM pg_stat_ssl WHERE pid = pg_backend_pid()")
        usuario, ssl, version = cur.fetchone()
    print(f"Cliente conectado al servidor como '{usuario}'. Cifrado SSL: {'sí (' + version + ')' if ssl
else 'no'}")
    demo(bd) if "--demo" in sys.argv else menu(bd)
    bd.close()


if __name__ == "__main__":
    main()
```

4.8 Cliente 4: Grafana

Grafana se configura automáticamente al arrancar: la fuente de datos apunta al servidor con el usuario de solo lectura, y el dashboard se carga desde un archivo JSON.

```
# Fuente de datos: Grafana se conecta al servidor como cliente de solo lectura
apiVersion: 1
datasources:
  - name: PostgreSQL IoT
    uid: pg-iot
    type: grafana-postgresql-datasource
    url: postgres:5432
    user: grafana_ro
    isDefault: true
    secureJsonData:
      password: grafana123
    jsonData:
```

```
      database: iotanalytics
      sslmode: require
      postgresVersion: 1600
      timescaledb: false
```

Ejemplo de consulta de un panel (temperatura por sensor en el rango de tiempo seleccionado):

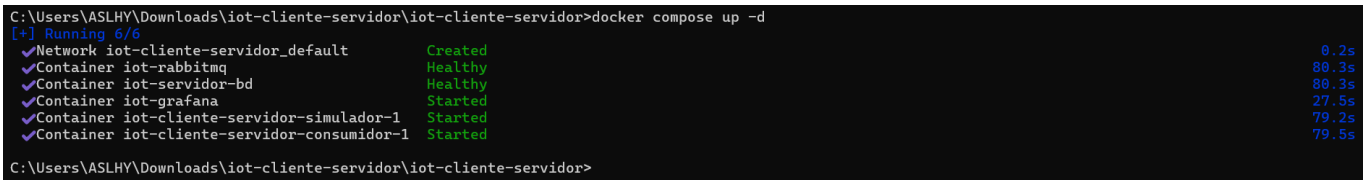
```
SELECT l.medido_en AS time, s.codigo AS metric, l.valor AS value
FROM lecturas l JOIN sensores s ON s.id = l.sensor_id
WHERE $__timeFilter(l.medido_en) AND s.tipo = 'temperatura'
ORDER BY 1
```

5. Demostración funcional

```
docker compose up -d --build
docker compose logs --tail 15 consumidor
docker compose run --rm cliente python cliente.py --demo
```

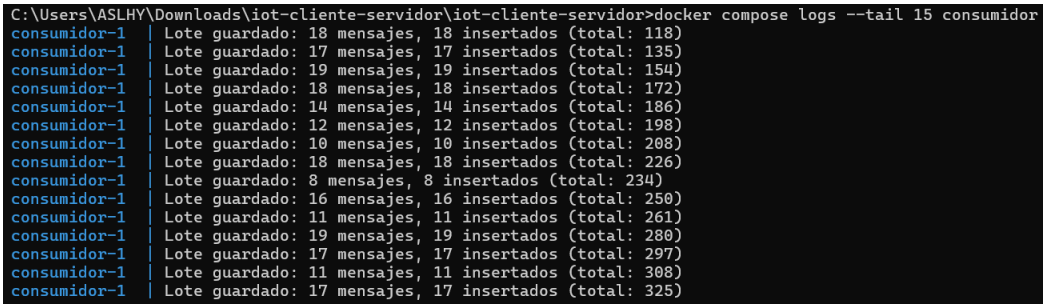
NO.	PRUEBA	RESULTADO ESPERADO	RESULTADO OBTENIDO
P1	Arranque de servidores y clientes	Todos los contenedores en ejecución	Correcto: PostgreSQL y RabbitMQ healthy; simulador, consumidor y Grafana iniciados
P2	Ingesta por lotes	El consumidor guarda lotes	Correcto: lotes de 8 a 19 mensajes; 325 lecturas acumuladas al momento de la captura
P3	Conexión segura del cliente	Cliente conectado con SSL y su propio usuario	Correcto: usuario 'operador', TLSv1.3
P4	Consultas del cliente	Dispositivos, últimas lecturas y resumen	Correcto: 3 dispositivos, 9 sensores con lecturas y unas 44 lecturas por sensor
P5	Lógica del servidor	El trigger genera alertas	Correcto: 5 alertas por temperaturas sobre el umbral
P6	Mensajería	La cola recibe y entrega mensajes	Correcto: 9.0 mensajes/s de entrada, entrega y confirmación
P7	Visualización	Dashboard actualizado en tiempo real	Correcto: 1,899 lecturas, 530 en el último minuto y 26 alertas

Evidencia 1 — P1: todos los contenedores en ejecución.



Servidores y clientes levantados con Docker Compose.

Evidencia 2 — P2: el consumidor guarda las lecturas en lotes y lleva el total acumulado.



Registro del consumidor guardando lotes.

Evidencia 3 — P3 y P4: el cliente se conecta con SSL y consulta los dispositivos y la última lectura de cada sensor.

```
C:\Users\ASLHY\Downloads\iot-cliente-servidor\iot-cliente-servidor>docker compose run --rm cliente python c
[+] Creating 1/1
 ✔Container iot-servidor-bd  Running
Cliente conectado al servidor como 'operador'. Cifrado SSL: sí (TLSv1.3)

--- Dispositivos registrados ---
CODIGO  NOMBRE              UBICACION  ESTADO  SENSORES
EMB-02  Embotelladora 2     Línea 3    activo  3
PRE-03  Prensa hidráulica 3 Línea 1    activo  3
TOR-01  Torno CNC 1         Línea 2    activo  3
(3 filas)

--- Última lectura de cada sensor ---
DISPOSITIVO  SENSOR      TIPO         VALOR  UNIDAD  UMBRAL_MAX  HORA
EMB-02       EMB02-ENE   energia      14.87  kWh     20.00       02:04:18
EMB-02       EMB02-TEMP  temperatura  81.00  °C      75.00       02:04:18
EMB-02       EMB02-VIB   vibracion    2.55   mm/s    5.00        02:04:18
PRE-03       PRE03-ENE   energia      21.45  kWh     30.00       02:04:19
PRE-03       PRE03-TEMP  temperatura  69.70  °C      85.00       02:04:19
PRE-03       PRE03-VIB   vibracion    4.77   mm/s    7.00        02:04:19
TOR-01       TOR01-ENE   energia      15.70  kWh     25.00       02:04:18
TOR-01       TOR01-TEMP  temperatura  61.71  °C      80.00       02:04:19
TOR-01       TOR01-VIB   vibracion    5.13   mm/s    6.00        02:04:18
```

Conexión y consultas del cliente.

Evidencia 4 — P4 y P5: resumen de la última hora y alertas generadas por el trigger del servidor.

```
--- Resumen de la última hora ---
SENSOR      LECTURAS  PROMEDIO  MINIMO  MAXIMO
EMB02-ENE   44        14.10     10.29   19.77
EMB02-TEMP  44        60.75     54.61   81.00
EMB02-VIB   44        2.98      1.89    3.95
PRE03-ENE   45        22.67     18.22   29.70
PRE03-TEMP  45        71.10     63.95   94.50
PRE03-VIB   45        4.56      3.37    5.87
TOR01-ENE   44        18.70     13.65   24.30
TOR01-TEMP  45        65.65     58.30   87.75
TOR01-VIB   44        3.90      3.18    5.15
(9 filas)

--- Alertas pendientes (generadas por el trigger del servidor) ---
ID  SENSOR      VALOR  UMBRAL  HORA
5   EMB02-TEMP  81.00  75.00   02:04:19
4   TOR01-TEMP  87.75  80.00   02:04:15
3   TOR01-TEMP  87.75  80.00   02:04:11
2   PRE03-TEMP  94.50  85.00   02:03:52
1   PRE03-TEMP  94.50  85.00   02:03:38
(5 filas)
```

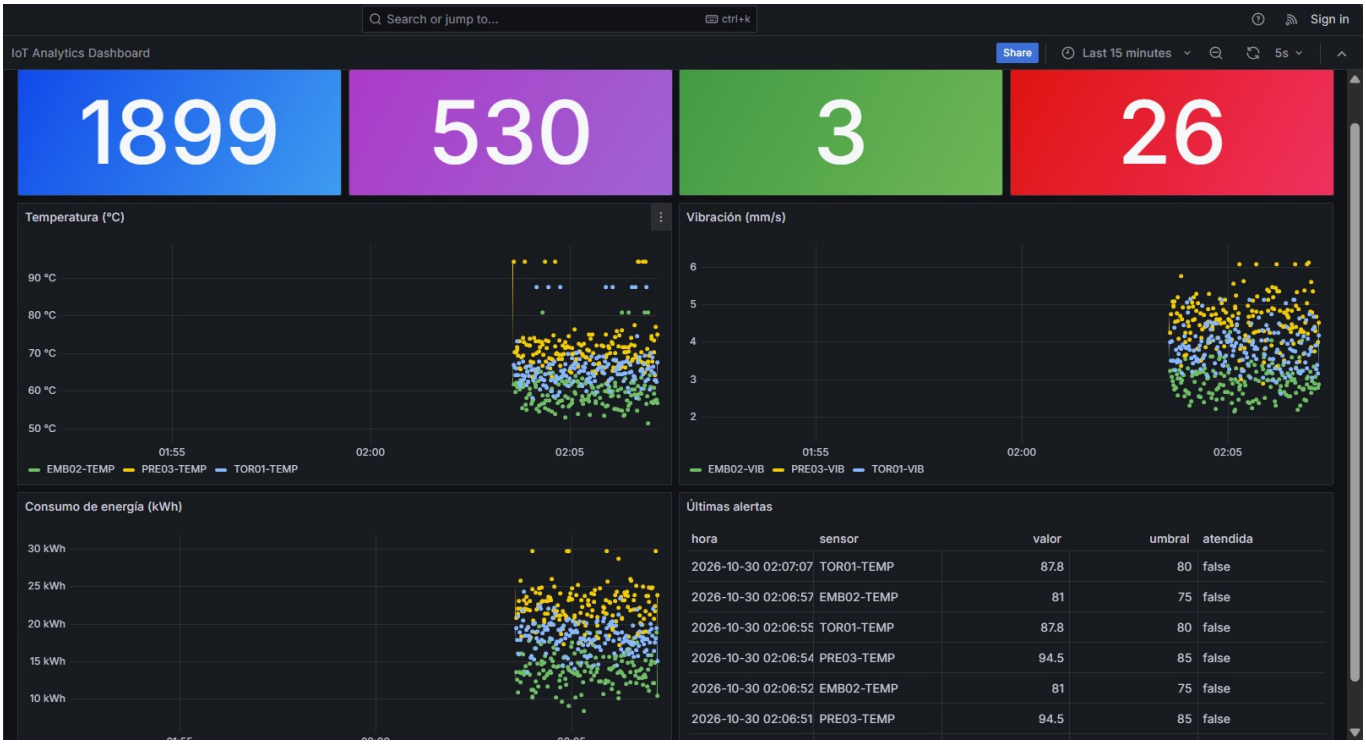
Resumen por sensor y alertas pendientes.

Evidencia 5 — P6: panel de RabbitMQ con la cola durable `lecturas` procesando 9 mensajes por segundo.



Cola lecturas en RabbitMQ.

Evidencia 6 — P7: dashboard de Grafana con las lecturas registradas, las del último minuto, los dispositivos activos, las alertas pendientes y las gráficas de temperatura, vibración y energía.

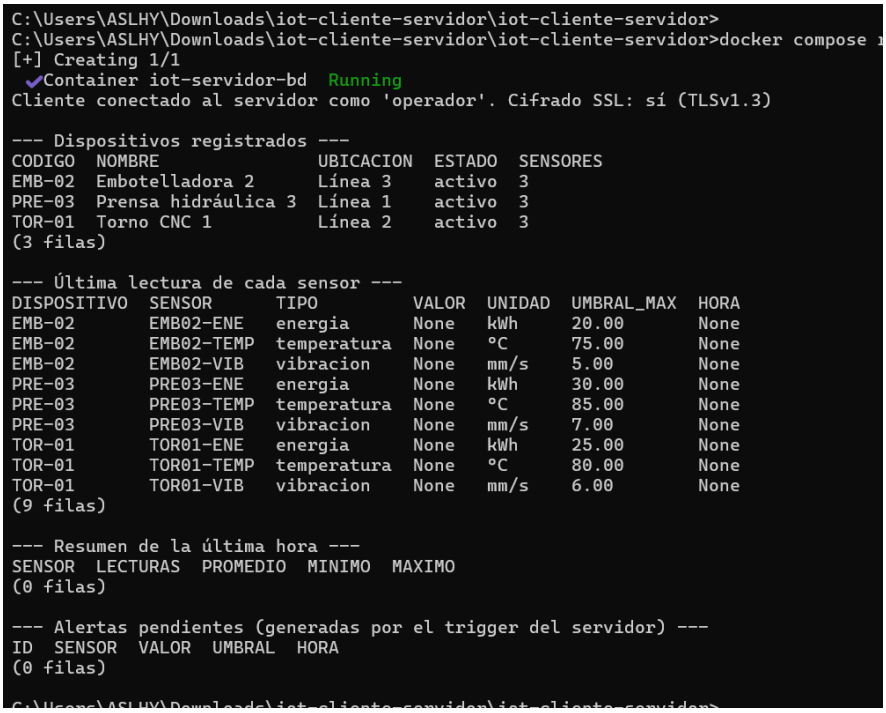


IoT Analytics Dashboard en Grafana.

6. Problemas encontrados y solución

En la primera ejecución, el contenedor de RabbitMQ se detuvo con error y los clientes que dependían de él no arrancaron.

```
[+] Running 7/7
 ✔iot-clientes:1.0                                Built                                                
```



El cliente funciona, pero sin lecturas mientras RabbitMQ está detenido.

INTENTO	CAMBIO	RESULTADO
1	Montar /var/lib/rabbitmq en memoria (tmpfs) con permisos abiertos, para que RabbitMQ creara la cookie desde cero	El error se repitió: el problema no era la carpeta, sino cómo Docker Desktop asigna permisos en esa computadora
2	Pasar la cookie como argumento (RABBITMQ_SERVER_ADDITIONAL_ERL_ARGS: -setcookie) y cambiar el chequeo de salud por una prueba del puerto 5672	RabbitMQ arrancó healthy y todo el sistema funcionó

Configuración final de RabbitMQ:

```
environment:
  RABBITMQ_SERVER_ADDITIONAL_ERL_ARGS: "-setcookie iotanalyticscookie"
healthcheck:
  test: ["CMD-SHELL", "bash -c '</dev/tcp/localhost/5672'"]
```

Prevención de otros conflictos. El proyecto usa un nombre propio en Docker Compose y puertos distintos a los de las tareas anteriores (5434, 5673, 15673 y 3001), para que no se mezcle con otros contenedores de la misma computadora.

7. Control de versiones y colaboración

La actividad se desarrolló de forma individual. El proyecto se publicó en un repositorio público: https://github.com/luisgael-ml/iot-cliente-servidor

```
git init
git add .
git commit -m "Sistema cliente-servidor IoT: PostgreSQL, RabbitMQ, clientes Python y Grafana"
git branch -M main
git remote add origin https://github.com/luisgael-ml/iot-cliente-servidor.git
git push -u origin main
```

Evidencia 7: el commit se registró a nombre de Luis Gael Monterrubio López y se publicó en GitHub.

```
C:\Users\ASLHY\Downloads\iot-cliente-servidor\iot-cliente-servidor>git commit -m "Sistema cliente-servidor IoT: PostgreSQL, RabbitMQ, clientes Python y Graf
ana"
[main (root-commit) 09c18dd] Sistema cliente-servidor IoT: PostgreSQL, RabbitMQ, clientes Python y Grafana
 13 files changed, 989 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 README.md
 create mode 100644 clientes/Dockerfile
 create mode 100644 clientes/cliente.py
 create mode 100644 clientes/conexiones.py
 create mode 100644 clientes/consumidor.py
 create mode 100644 clientes/requirements.txt
 create mode 100644 clientes/simulador.py
 create mode 100644 docker-compose.yml
 create mode 100644 grafana/dashboards/iot-analytics.json
 create mode 100644 grafana/provisioning/dashboards/dashboards.yml
 create mode 100644 grafana/provisioning/datasources/postgres.yml
 create mode 100644 servidor/init.sql

C:\Users\ASLHY\Downloads\iot-cliente-servidor\iot-cliente-servidor>git branch -M main

C:\Users\ASLHY\Downloads\iot-cliente-servidor\iot-cliente-servidor>git remote add origin https://github.com/luisgael-ml/iot-cliente-servidor.git

C:\Users\ASLHY\Downloads\iot-cliente-servidor\iot-cliente-servidor>git push -u origin main
Enumerating objects: 22, done.
Counting objects: 100% (22/22), done.
Delta compression using up to 4 threads
Compressing objects: 100% (16/16), done.
Writing objects: 100% (22/22), 11.57 KiB | 215.00 KiB/s, done.
Total 22 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/luisgael-ml/iot-cliente-servidor.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Publicación del proyecto en GitHub.

7.1 Reflexión sobre la colaboración

Preparé el sistema para compartirlo y discutirlo. Con un solo comando, cualquier compañero levanta el sistema completo y puede ver en Grafana y RabbitMQ cómo interactúan los clientes y los servidores, sin instalar nada más que Docker.

Documenté el problema de RabbitMQ con su diagnóstico y los dos intentos de solución. Es un error que otros pueden encontrar en Docker Desktop, y tenerlo explicado les ahorra tiempo. El sistema también reutiliza las ideas de las tareas 3 y 4 (SSL, transacciones, usuarios con permisos mínimos), lo que permite comparar cómo se aplican en Java, C# y Python.

8. Conclusiones

El modelo cliente-servidor centraliza los datos y las reglas en el servidor, mientras los clientes se dedican a pedir y presentar información. En este proyecto, PostgreSQL concentró los datos, la seguridad y la generación de alertas, y cuatro clientes distintos lo usaron al mismo tiempo, cada uno con sus propios permisos.

Agregar RabbitMQ convirtió la arquitectura en n capas: los sensores quedaron desacoplados de la base de datos, la ingesta se hizo por lotes con transacciones y no se perdieron mensajes. La falla de RabbitMQ demostró esa independencia, porque el servidor y el cliente de consultas siguieron funcionando.

Docker permitió levantar todo el sistema con un comando. Grafana mostró en tiempo real lo que ocurría en el servidor, cerrando el ciclo desde el sensor hasta el usuario.

Referencias

PostgreSQL Global Development Group. (s.f.). *Architectural fundamentals*. https://www.postgresql.org/docs/16/tutorial-arch.html

Broadcom. (s.f.). *RabbitMQ documentation*. https://www.rabbitmq.com/docs

Grafana Labs. (s.f.). *PostgreSQL data source*. https://grafana.com/docs/grafana/latest/datasources/postgres/

Docker Inc. (s.f.). *Docker Compose*. https://docs.docker.com/compose/

Psycopg. (s.f.). *Psycopg 3 documentation*. https://www.psycopg.org/psycopg3/docs/